

Lab10_WebFlux_WebClient

Code ทุก TODO

- Repository

```
// — 1. หา Product 1 รายการ —————
/**
 * TODO: คืน Mono<Product> จาก store โดยใช้ id
 *       ถ้าไม่พบให้คืน Mono.empty()
 *
 * Hint: store.get(id) คืน Product หรือ null
 *       ถ้า null ให้ใช้ Mono.empty()
 *       ถ้ามีค่าให้ใช้ Mono.just(product)
 */
public Mono<Product> findById(String id) {

    Product product = store.get(id);

    if (product == null) {
        return Mono.empty();
    }

    return Mono.just(product);
}

// — 2. หา Product ทั้งหมด —————
/**
 * TODO: คืน Flux<Product> ของทุกรายการใน store
 *
 * Hint: store.values() คืน Collection<Product>
 *       ใช้ Flux.fromIterable(...) แปลงเป็น Flux
 */
public Flux<Product> findAll() {

    return Flux.fromIterable(store.values());
}
```

```

// — 3. บันทึก Product —————
/**
 * TODO: บันทึก product ลง store แล้วคืน Mono<Product>
 *
 * Hint: store.put(product.getId(), product)
 *       แล้วใช้ Mono.just(product) คืนค่า
 */
public Mono<Product> save(Product product) {

    store.put(product.getId(), product);

    return Mono.just(product);
}

// — 4. ลบ Product —————
/**
 * TODO: ลบ product จาก store แล้วคืน Mono<Void>
 *
 * Hint: store.remove(id)
 *       แล้วใช้ Mono.empty() คืนค่า (Mono<Void>)
 */
public Mono<Void> deleteById(String id) {

    store.remove(id);

    return Mono.empty();
}

// — 5. กรองตาม category —————
/**
 * TODO: คืน Flux<Product> ที่ category ตรงกัน
 *
 * Hint: findAll()
 *       .filter(p -> p.getCategory().equalsIgnoreCase(category))
 */
public Flux<Product> findByCategory(String category) {

    return findAll()
        .filter(p -> p.getCategory().equalsIgnoreCase(category));
}
}

```

- Service

```
// — 1. ดึง Product 1 รายการ —————
/**
 * TODO: เรียก repository.findById(id) แล้วคืนผล
 * ถ้าไม่พบให้ throw RuntimeException("Product not found: " + id)
 *
 * Hint: repository.findById(id)
 *       .switchIfEmpty(Mono.error(new RuntimeException(...)))
 */
public Mono<Product> getById(String id) {

    return repository.findById(id)
        .switchIfEmpty(
            Mono.error(
                new RuntimeException("Product not found: " + id)
            )
        );
}

// — 2. ดึง Product ทั้งหมด —————
/**
 * TODO: เรียก repository.findAll() แล้วคืนผล
 */
public Flux<Product> getAll() {

    return repository.findAll();
}

// — 3. บันทึก Product —————
/**
 * TODO: เรียก repository.save(product) แล้วคืนผล
 *
 * เพิ่มเติม: ถ้า product.getId() เป็น null ให้ generate id ใหม่
 * Hint: java.util.UUID.randomUUID().toString()
 */
public Mono<Product> save(Product product) {

    if (product.getId() == null) {
        product.setId(java.util.UUID.randomUUID().toString());
    }

    return repository.save(product);
}
```

```

// — 4. ลบ Product —————
/**
 * TODO: เรียก repository.deleteById(id) แล้วคืนผล
 */
public Mono<Void> delete(String id) {

    return repository.deleteById(id);

}

// — 5. กรองตาม category —————
/**
 * TODO: เรียก repository.findByCategory(category) แล้วคืนผล
 */
public Flux<Product> getByCategory(String category) {

    return repository.findByCategory(category);

}

// — 6. คำนวณราคาหลังส่วนลด —————
/**
 * TODO: หา Product จาก id แล้วคืน discountedPrice
 *
 * Hint: getById(id)
 *        .map(p -> p.getDiscountedPrice())
 */
public Mono<Double> getDiscountedPrice(String id) {

    return getById(id)
        .map(p -> p.getDiscountedPrice());

}
    You, 17 hours ago • Uncommitted changes
}

```

- Controller

```
24 @RestController
25 @RequestMapping("/products")
26 public class ProductController {
27
28     // — Constructor Injection (DIP - SOLID) —————
29     private final ProductService service;
30
31     public ProductController(ProductService service) {
32         this.service = service;
33     }
34
35     // =====
36     // ✅ ตัวอย่างที่ทำเสร็จแล้ว - ศึกษาแล้วทำ endpoint ที่เหลือ
37     // =====
38
39     /**
40      * GET /products/{id}
41      * คืน Mono<Product> - ค้นหา Product 1 รายการ
42      *
43      * ทดสอบ: GET http://localhost:8080/products/1
44      */
45     @GetMapping("/{id}")
46     public Mono<Product> getById(@PathVariable String id) {
47         return service.getById(id);
48     }
49
50     // =====
51     // ❌ TODO: เติม method body ด้านล่างนี้
52     // =====
53
54     /**
55      * GET /products
56      * TODO: คืน Flux<Product> ทุกรายการ
57      *
58      * Hint: เรียก service.getAll()
59      * ทดสอบ: GET http://localhost:8080/products
60      */
61     @GetMapping
62     public Flux<Product> getAll() {
63
64         return service.getAll();
65
66     }
67 }
```

```

    * POST /products
    * TODO: รับ Product จาก request body แล้วบันทึก
    *
    * Hint: เรียก service.save(product)
    * ทดสอบ: POST http://localhost:8080/products
    *   Body: { "name": "...", "price": 999.0, ... }
    */
    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {

        return service.save(product);

    }

    /**
     * DELETE /products/{id}
     * TODO: ลบ Product และคืน Mono<Void>
     *
     * Hint: เรียก service.delete(id)
     * ทดสอบ: DELETE http://localhost:8080/products/1
     */
    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) {

        return service.delete(id);

    }

    /**
     * GET /products/category/{category}
     * TODO: คืน Flux<Product> ที่กรองตาม category
     *
     * Hint: เรียก service.getByCategory(category)
     * ทดสอบ: GET http://localhost:8080/products/category/Electronics
     */
    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(
        @PathVariable String category) {

        return service.getByCategory(category);

    }

    Narathip Phromprakai, last week • update

    /**
     * GET /products/{id}/price
     * TODO: คืน Mono<Double> ราคาหลังส่วนลด
     *
     * Hint: เรียก service.getDiscountedPrice(id)
     * ทดสอบ: GET http://localhost:8080/products/1/price
     */
    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(
        @PathVariable String id) {

        return service.getDiscountedPrice(id);

    }
}

```

- WebClient

```
24 public class ProductWebClient {
25
26     /**
27      * GET /products → Flux<Product>
28      * TODO: ดึง Product ทั้งหมดจาก server
29      *
30      * Hint: client.get()
31      *         .uri("/products")
32      *         .retrieve()
33      *         .bodyToFlux(Product.class)
34      */
35     public Flux<Product> getAllProducts() {
36
37         return client.get()
38             .uri(uri: "/products")
39             .retrieve()
40             .bodyToFlux(elementClass: Product.class);
41     }
42
43     /**
44      * POST /products → Mono<Product>
45      * TODO: ส่ง Product ใหม่ไปยัง server
46      *
47      * Hint: client.post()
48      *         .uri("/products")
49      *         .bodyValue(product) + ส่ง body
50      *         .retrieve()
51      *         .bodyToMono(Product.class)
52      */
53     public Mono<Product> createProduct(Product product) {
54
55         return client.post()
56             .uri(uri: "/products")
57             .bodyValue(product)
58             .retrieve()
59             .bodyToMono(elementClass: Product.class);
60     }
61
62     /**
63      * DELETE /products/{id} → Mono<Void>
64      * TODO: ส่ง request au Product
65      *
66      * Hint: client.delete()
67      *         .uri("/products/{id}", id)
68      *         .retrieve()
69      *         .bodyToMono(Void.class)
70      */
71     public Mono<Void> deleteProduct(String id) {
72
73         return client.delete()
74             .uri(uri: "/products/{id}", id)
75             .retrieve()
76             .bodyToMono(elementClass: Void.class);
77     }
78 }
```

```

/**
 * GET /products/category/{category} → Flux<Product>
 * TODO: ดึง Product ตาม category
 *
 * Hint: client.get()
 *       .uri("/products/category/{category}", category)
 *       .retrieve()
 *       .bodyToFlux(Product.class)
 */
public Flux<Product> getByCategory(String category) {

    return client.get()
        .uri(uri: "/products/category/{category}", category)
        .retrieve()
        .bodyToFlux(elementClass: Product.class);

}

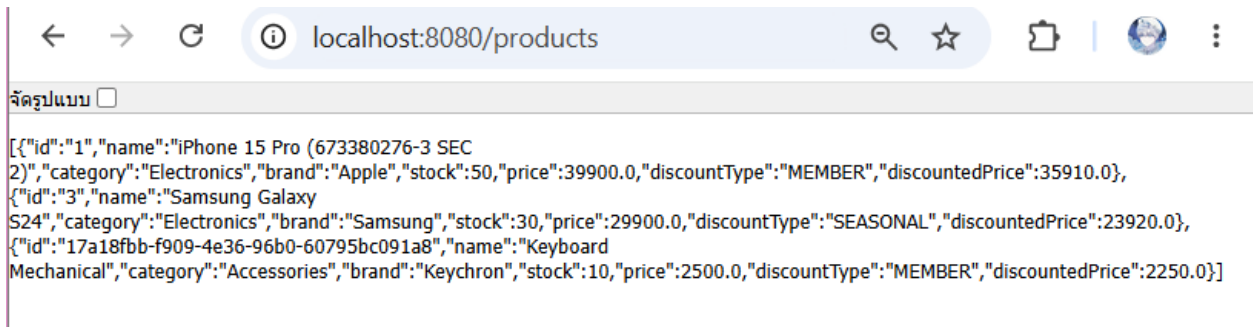
/**
 * GET /products/{id}/price → Mono<Double>
 * TODO: ดึงราคาหลังส่วนลด
 *       แล้ว chain .doOnNext() เพื่อ log ราคาที่ได้
 *
 * Hint: client.get()
 *       .uri("/products/{id}/price", id)
 *       .retrieve()
 *       .bodyToMono(Double.class)
 *       .doOnNext(price -> System.out.println("Price: " + price))
 */
public Mono<Double> getDiscountedPrice(String id) {

    return client.get()
        .uri(uri: "/products/{id}/price", id)
        .retrieve()
        .bodyToMono(elementClass: Double.class)
        .doOnNext(
            price -> System.out.println(
                "Price: " + price
            )
        );
}

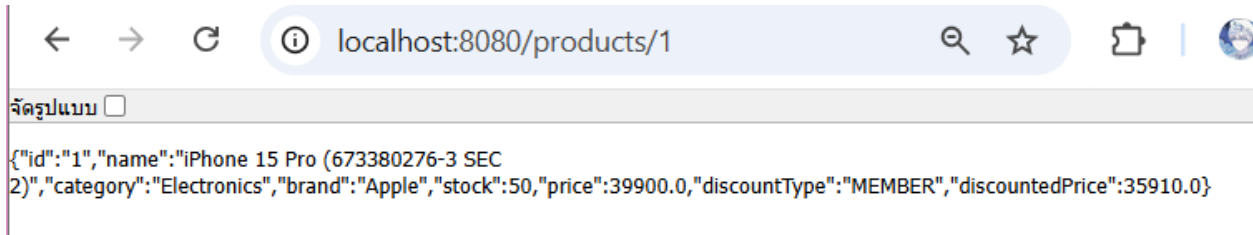
```

ทดสอบทุก endpoint

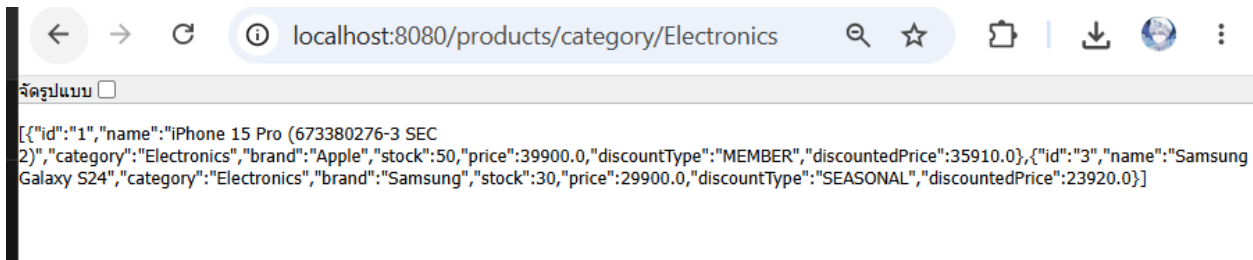
ดูสินค้าทั้งหมด — GET



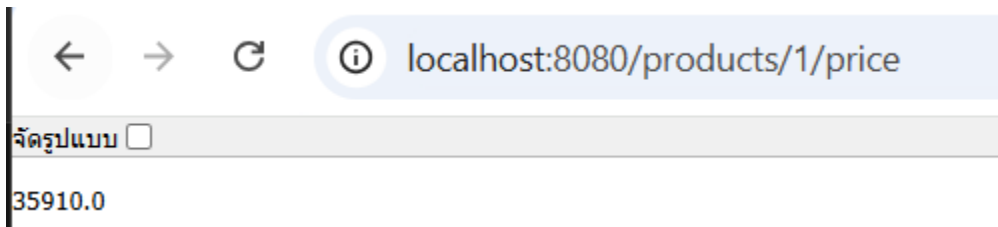
ดูสินค้าตาม ID — GET



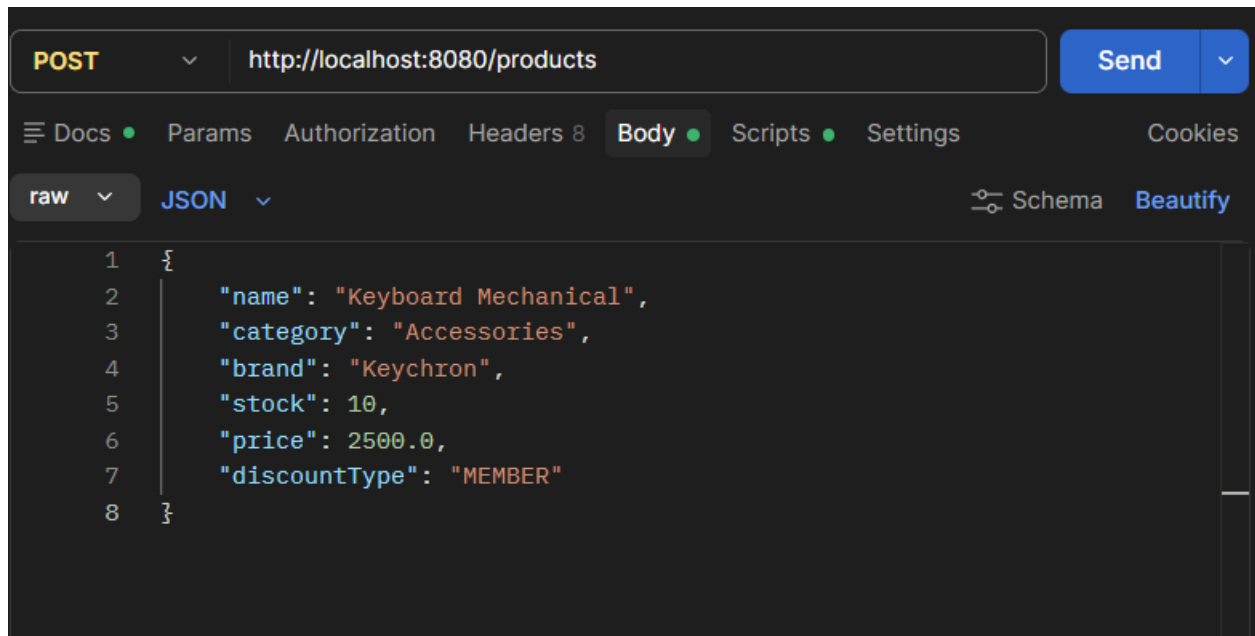
ค้นหาตาม Category — GET



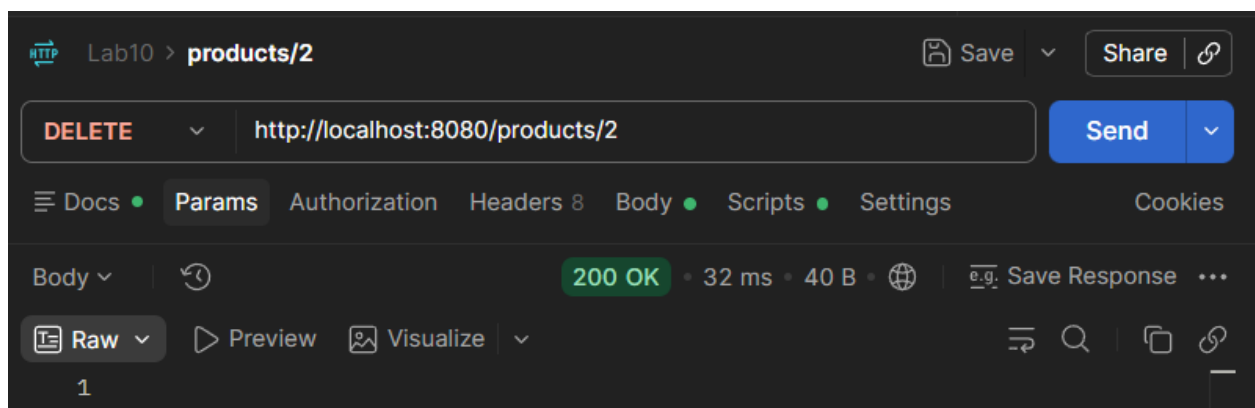
price — GET



สร้างสินค้า — POST



ลบสินค้า — DELETE



Lab10 > products/2

GET http://localhost:8080/products

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body 200 OK • 18 ms • 609 B • Save Response

{ } JSON Preview Visualize

```
1  [
2    {
3      "id": "1",
4      "name": "iPhone 15 Pro (673380276-3 SEC 2)",
5      "category": "Electronics",
6      "brand": "Apple",
7      "stock": 50,
8      "price": 39900.0,
9      "discountType": "MEMBER",
10     "discountedPrice": 35910.0
11   },
12   {
13     "id": "3",
14     "name": "Samsung Galaxy S24",
15     "category": "Electronics",
16     "brand": "Samsung",
17     "stock": 30,
18     "price": 29900.0,
19     "discountType": "SEASONAL",
20     "discountedPrice": 23920.0
21   },
22   {
23     "id": "17a18fbb-f909-4e36-96b0-60795bc091a8",
24     "name": "Keyboard Mechanical",
25     "category": "Accessories",
26     "brand": "Keychron",
27     "stock": 10,
28     "price": 2500.0,
29     "discountType": "MEMBER",
30     "discountedPrice": 2250.0
  }
```

ตอบคำถาม

1.Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ Reactive Programming ช่วยให้ระบบสามารถจัดการงานที่ต้องรอ I/O เช่น API หรือ Database ได้โดยไม่ต้อง Block Thread ทำให้รองรับงานพร้อมกันได้ดีขึ้น

2.Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ - Mono<T> ใช้สำหรับข้อมูล 0 หรือ 1 ค่า เพราะการค้นหาด้วย ID จะได้พบ → Product 1 ตัว, ไม่พบ → 0 ตัวจึงเหมาะกับ Mono<Product>

- Flux<T> ใช้สำหรับข้อมูล 0 ถึงหลายค่า เพราะ findAll() สามารถคืน Product ได้หลายตัวจึงใช้ Flux<Product>

3.WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ

- .get() ใช้กำหนดว่าเราต้องการส่ง HTTP Request แบบ GET
- .uri() ใช้กำหนด URL หรือ Endpoint ที่ต้องการเรียก
- .retrieve() ใช้สำหรับส่ง Request และเตรียมรับ Response จาก Server
- .bodyToMono(Product.class) ใช้แปลง Response Body ที่ได้รับมาเป็น Product จำนวน 0-1 Object
- .bodyToFlux(Product.class) ใช้เมื่อ Response มีข้อมูล Product หลายรายการ

4.Code ของ Repository, Service, Controller พร้อม Comment

ตอบ

@Repository

```
public class ProductRepository {
```

```
    // เก็บข้อมูล Product โดยใช้ id เป็น Key
```

```
    private final Map<String, Product> store = new ConcurrentHashMap<>();
```

```
    // ค้นหา Product จาก ID
```

```
    public Mono<Product> findById(String id) {
```

```
        // ดึง Product จาก Map
```

```
        Product product = store.get(id);
```

```
        // ถ้าไม่พบข้อมูล ให้คืน Mono ว่าว่าง
```

```
        if (product == null) {
            return Mono.empty();
        }
```

```
        // ถ้าพบ Product ให้คืนข้อมูลแบบ Mono
```

```
        return Mono.just(product);
    }
}
```

// ค้นหา Product ทั้งหมด

```
public Flux<Product> findAll() {
```

// แปลงข้อมูลทั้งหมดใน Map เป็น Flux

```
    return Flux.fromIterable(store.values());
}
```

// บันทึก Product

```
public Mono<Product> save(Product product) {
```

// เก็บ Product ลงใน Map

```
    store.put(product.getId(), product);
```

// คืน Product ที่บันทึกแล้ว

```
    return Mono.just(product);
}
```

// ลบ Product ตาม ID

```
public Mono<Void> deleteById(String id) {
```

// ลบข้อมูลจาก Map

```
    store.remove(id);
```

// ไม่มีข้อมูลที่จะส่งกลับ

```
    return Mono.empty();
}
```

// ค้นหา Product ตาม Category

```
public Flux<Product> findByCategory(String category) {
```

// ค้นหา Product ทั้งหมด

// แล้ว filter เฉพาะ Category ที่ตรงกัน

```
    return findAll()
        .filter(p ->
            p.getCategory().equalsIgnoreCase(category)
        );
}
```

```
}  
}
```

@Service

```
public class ProductService {
```

```
    private final ProductRepository repository;
```

```
    // Constructor Injection
```

```
    public ProductService(ProductRepository repository) {  
        this.repository = repository;  
    }
```

```
    // ค้นหา Product ตาม ID
```

```
    public Mono<Product> getById(String id) {
```

```
        return repository.findById(id)
```

```
        // ถ้าไม่พบ Product ให้สร้าง Error
```

```
        .switchIfEmpty(  
            Mono.error(  
                new RuntimeException(  
                    "Product not found: " + id  
                )  
            )  
        );  
    }
```

```
    // ดึง Product ทั้งหมด
```

```
    public Flux<Product> getAll() {  
        return repository.findAll();  
    }
```

```
    // บันทึก Product
```

```
    public Mono<Product> save(Product product) {
```

```
        // ถ้ายังไม่มี ID ให้สร้าง ID ใหม่
```

```
        if (product.getId() == null) {  
            product.setId(  
                // ...  
            )  
        }
```

```

        java.util.UUID.randomUUID().toString()
    );
}

// ส่ง Product ไปให้ Repository บันทึก
return repository.save(product);
}

// ลบ Product
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

// ค้นหา Product ตาม Category
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// คำนวณราคาหลังส่วนลด
public Mono<Double> getDiscountedPrice(String id) {

    // ค้นหา Product ก่อน
    return getById(id)

        // เมื่อได้ Product แล้ว
        // เรียก getDiscountedPrice()
        .map(p -> p.getDiscountedPrice());
}
}

```

@RestController

@RequestMapping("/products")

public class ProductController {

private final ProductService service;

// Constructor Injection

public ProductController(ProductService service) {

```
        this.service = service;
    }
}
```

```
// GET /products/{id}
```

```
// ค้นหา Product ตาม ID
```

```
@GetMapping("/{id}")
```

```
public Mono<Product> getById(
```

```
    @PathVariable String id) {
```

```
    return service.getById(id);
```

```
}
```

```
// GET /products
```

```
// ค้นหา Product ทั้งหมด
```

```
@GetMapping
```

```
public Flux<Product> getAll() {
```

```
    return service.getAll();
```

```
}
```

```
// POST /products
```

```
// เพิ่ม Product ใหม่
```

```
@PostMapping
```

```
public Mono<Product> save(
```

```
    @RequestBody Product product) {
```

```
    return service.save(product);
```

```
}
```

```
// DELETE /products/{id}
```

```
// ลบ Product ตาม ID
```

```
@DeleteMapping("/{id}")
```

```
public Mono<Void> delete(
```

```
    @PathVariable String id) {
```

```
    return service.delete(id);
```

```
}
```

```
// GET /products/category/{category}
```

```
// ค้นหา Product ตาม Category
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(
    @PathVariable String category) {

    return service.getByCategory(category);
}

// GET /products/{id}/price
// แสดงราคาหลังส่วนลด
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(
    @PathVariable String id) {

    return service.getDiscountedPrice(id);
}
}
```